Relatório Técnico de Trabalho Prático

Compiladores: Interpretador e Front-end para Subset de Ada

David Sá (up202303508)
João Morais (up202307077)

Outubro 2025

Conteúdo

1 Introdução **2**

2 Método e Instrumentos **2**
2.1 Fluxo de Compilação e Interpretação 2
2.2 Estruturas de Dados Chave 2
2.2.1 Árvore Sintática Abstrata (AST) 2
2.2.2 Tabela de Símbolos 2

3 Implementação do Subset de Ada **3**
3.1 Estrutura do Programa e Declarações 3
3.2 Comandos e Controlo de Fluxo 3
3.3 Expressões 3
3.4 Análise Semântica e Verificação de Erros 3

4 Instruções de Compilação e Execução **4**

5 Conclusão **4**

1 Introdução

Este documento técnico descreve a realização da primeira atividade prática da disciplina de Compiladores (DCC-FCUP). O objetivo foi desenvolver um **interpretador funcional** para uma versão simplificada da linguagem de programação Ada.

O projeto engloba o *front-end* completo de um compilador — o Analisador Léxico (Scanner) e o Analisador Sintático (Parser) — mas vai mais além, incluindo uma fase robusta de **Análise Semântica** e um **Interpretador** que executa a Árvore Sintática Abstrata (AST) gerada.

2 Método e Instrumentos

O projeto foi implementado em linguagem C, utilizando as ferramentas padrão do ecossistema GNU: `Flex` para a análise lexical e `Bison` para a análise sintática e semântica.

2.1 Fluxo de Compilação e Interpretação

O sistema segue um fluxo de 5 etapas:

1. **Análise Léxica (Scanner):** O `scanner.flex` lê o código-fonte e o transforma numa sequência de *tokens* (e.g., `IF`, `ID`, `INT`).
2. **Análise Sintática (Parser):** O `parser.bison` utiliza a gramática definida para verificar a correção sintática da sequência de *tokens*.
3. **Análise Semântica (Parser):** Integrada nas ações do `Bison`, esta fase verifica a lógica do programa. Utiliza a Tabela de Símbolos para validar declarações e realiza uma **verificação de tipos** rigorosa.
4. **Geração da AST:** À medida que o Parser valida as regras, as ações semânticas constroem a Árvore Sintática Abstrata (`ast.c`, `ast.h`).
5. **Interpretação (Runtime):** O `interpreter.c` percorre a AST (função `execCmd`) e avalia as suas expressões (função `eval`) para executar o programa.

2.2 Estruturas de Dados Chave

2.2.1 Árvore Sintática Abstrata (AST)

A AST (`ast.h`) é a estrutura central que representa a lógica do programa. É composta por nós de `Expr` (Expressão) e `Command` (Comando), eliminando pormenores sintáticos (como parêntesis ou ponto e vírgulas) para focar-se no significado.

2.2.2 Tabela de Símbolos

Implementada em `symboltable.c`, esta estrutura é vital para a **análise semântica**.

- É uma tabela hash com chaining para resolver colisões.
- Utiliza um mecanismo de pilha de scopes (embora nesta fase se utilize primariamente o scope global).
- Permite a verificação imediata de variáveis não declaradas (`symtab_lookup()`).
- Previne redeclaraçõe no mesmo scope (`symtab_insert()`).
- É a fonte de informação para a verificação de tipos no parser.

3 Implementação do Subset de Ada

O Parser (`parser.bison`) foi concebido para suportar na íntegra o *subset* de Ada definido, com foco principal na correção semântica.

3.1 Estrutura do Programa e Declarações

A regra principal `program` segue a sintaxe exigida:

```
PROCEDURE MAIN IS
  <decls>
BEG
  <cmd_list>
END MAIN;
```

Listing 1: Estrutura Base do Programa

As variáveis são inseridas (`symtab_insert`) na Tabela de Símbolos durante a fase `decls`, garantindo que estão prontas para serem validadas na lista de comandos.

3.2 Comandos e Controlo de Fluxo

O Parser suporta os comandos de controlo de fluxo pedidos, gerando o nó `Command` apropriado na AST:

- **Atribuição:** `ID := expr ;` (gera o nó `E_ASSIGNMENT`).
- **Condicional:** Suporta `IF...THEN...END IF`, `IF...THEN...ELSE...END IF` e a sintaxe `ELSIF` para *else-ifs* encadeados (gera `IF_CMD` ou `IF_ELSE_CMD`).
- **Loop:** `WHILE expr LOOP cmd_list END LOOP ;` (gera `WHILE_CMD`).
- **I/O:** `PUT_LINE expr ;` e `GET_LINE ID ;` (gera `PUT_LINE_CMD` e `GET_LINE_CMD`).

3.3 Expressões

As expressões aritméticas e booleanas são processadas com a devida consideração da precedência de operadores:

- Foi utilizada a declaração `%left` no `parser.bison` para resolver os conflitos *shift/reduce* e definir a ordem de avaliação correta (`MULT/DIV` antes de `PLUS/MINUS`).
- As expressões são mapeadas para os nós `E_OPERATION` na AST.

3.4 Análise Semântica e Verificação de Erros

Esta foi a componente central do desenvolvimento e **onde foram aplicadas as melhorias mais críticas**. O `parser.bison` implementa um conjunto rigoroso de verificações semânticas em tempo de compilação, e o `interpreter.c` valida erros de runtime.

- **Verificação de Tipos (Compile-time):** O parser foi melhorado para recusar programas logicamente inválidos que antes eram aceites:
  - `My_Int := TRUE;` falha (atribuição de tipos incompatíveis).
  - `IF 5 THEN...` falha (condição de `IF` tem de ser `Boolean`).
  - Foram adicionadas verificações de tipo rigorosas para impedir comparações inválidas (e.g., `IF 5 > TRUE THEN...`) ou operações inválidas (e.g., `IF TRUE > FALSE THEN...`).

  - Inicialmente, `Get_Line` aceitava qualquer variável; foi corrigido para falhar (e.g., `Get_Line(My_Bool);`) se a variável não for `Integer`.

- **Verificação de Declarações (Compile-time):** O uso de qualquer variável não declarada numa expressão ou atribuição causa um erro semântico, pois o `symtab_lookup()` falha.

- **Verificação de Erros (Runtime):** O interpretador, que inicialmente falhava silenciosamente ou crashava, foi modificado para detetar erros de runtime:

  - `My_Var := 5 / 0;` compila, mas o `interpreter.c` deteta a **divisão por zero** e termina o programa com um erro explícito.
  - A avaliação de `TRUE` e `FALSE` literais em condições (e.g. `IF TRUE THEN...`) foi corrigida para funcionar como esperado.

4 Instruções de Compilação e Execução

O trabalho será submetido num ficheiro `COMP-TP1_GRUPO_n.zip`, contendo todo o código-fonte (`.c`, `.h`, `scanner.flex`, `parser.bison`) e este relatório (`README.pdf`).

O projeto inclui um `Makefile` para facilitar a compilação.

Para compilar e executar o projeto:

```
# 1. Compilar o projeto (gera o executavel "interpreter")
make

# 2. Executar o interpretador com um ficheiro de teste
./interpreter <nome_do_ficheiro>.ada
```

Foram criados vários ficheiros de teste (`test1.ada`, `test2.ada`, etc.). Para os compilar e executar rapidamente:

```
# 1. (OPCIONAL) Apagar os objetos e ficheiros intermedios
make clean

# 2. Compilar e executar o ficheiro testX.ada
# (Substituir X pelo numero do teste)
make testX
```

5 Conclusão

A primeira fase do trabalho de Compiladores está funcionalmente concluída. O *front-end* (Scanner e Parser) foi efetivamente implementado, mas o projeto foi mais longe, assegurando uma camada de análise semântica que foi significativamente robustecida durante o desenvolvimento.

Além do solicitado (a AST), foi implementado um interpretador que valida a correção da árvore e executa o programa.

Igualmente importante, o código foi ativamente revisto e depurado para corrigir falhas críticas que foram identificadas, como o ***crash* no `Put_Line` com strings** e diversas fugas de memória na gestão de identificadores. O resultado é uma aplicação estável, correta e que cumpre todos os requisitos do enunciado.